AI Lab 3

*A**

Case 1 : 1. g(n): Depth of the node, h(n): Number of Misplaced tiles

```
import heapq

GOAL = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)


def misplaced_tiles(state):
    count = 0

    for i in range(9):
        if state[i] != 0 and state[i] != GOAL[i]:
            count += 1

    return count


def get_neighbors(state):
    neighbors = []

    zero_index = state.index(0)
    row = zero_index // 3
    col = zero_index % 3

    moves = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]

    for dr, dc in moves:
        new_row = row + dr
```

```
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:

            new_index = new_row * 3 + new_col

            new_state = list(state)

            new_state[zero_index], new_state[new_index] = \
                new_state[new_index], new_state[zero_index]

            neighbors.append(tuple(new_state))

    return neighbors


def print_puzzle(state):
    for i in range(0, 9, 3):
        print(" ".join(map(str, state[i:i + 3])))
    print()


def reconstruct_path(parent, state):
    path = []

    while state is not None:
        path.append(state)
        state = parent[state]

    path.reverse()

    return path


def a_star(initial):

    priority_queue = []

    g_cost = {initial: 0}
    parent = {initial: None}
```

```
h = misplaced_tiles(initial)
f = h

heapq.heappush(priority_queue, (f, 0, initial))

visited = set()

while priority_queue:

    f, g, current = heapq.heappop(priority_queue)

    if current in visited:
        continue

    visited.add(current)

    if current == GOAL:
        return reconstruct_path(parent, current)

    for neighbor in get_neighbors(current):

        if neighbor in visited:
            continue

        new_g = g + 1

        if neighbor not in g_cost or new_g < g_cost[neighbor]:

            g_cost[neighbor] = new_g
            parent[neighbor] = current

            h = misplaced_tiles(neighbor)
            f = new_g + h

            heapq.heappush(
                priority_queue,
                (f, new_g, neighbor)
            )
```

```
    return None


initial_state = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
)

print("INITIAL STATE:")
print_puzzle(initial_state)

print("GOAL STATE:")
print_puzzle(GOAL)

solution = a_star(initial_state)

if solution:

    print("SOLUTION STEPS:")
    print()

    for step, state in enumerate(solution):
        print("Step", step)
        print_puzzle(state)

    print("Total moves:", len(solution) - 1)

else:
    print("No solution exists.")
```

```
INITIAL STATE:
1 2 3
4 0 6
7 5 8

GOAL STATE:
1 2 3
4 5 6
7 8 0

SOLUTION STEPS:

Step 0
1 2 3
4 0 6
7 5 8

Step 1
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0

Total moves: 2
```

Case 2 : 2. g(n): Depth of the node, h(n): Manhattan Distance

```
import heapq

GOAL = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)
```

```
def manhattan_distance(state):
    distance = 0

    for i in range(9):

        if state[i] == 0:
            continue

        current_row = i // 3
        current_col = i % 3

        goal_index = GOAL.index(state[i])
        goal_row = goal_index // 3
        goal_col = goal_index % 3

        distance += abs(current_row - goal_row)
        distance += abs(current_col - goal_col)

    return distance


def get_neighbors(state):
    neighbors = []

    zero_index = state.index(0)

    row = zero_index // 3
    col = zero_index % 3

    moves = [
        (-1, 0),
        (1, 0),
        (0, -1),
        (0, 1)
    ]

    for dr, dc in moves:
```

```
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:

            new_index = new_row * 3 + new_col

            new_state = list(state)

            new_state[zero_index], new_state[new_index] = \
                new_state[new_index], new_state[zero_index]

            neighbors.append(tuple(new_state))

    return neighbors


def print_puzzle(state):
    for i in range(0, 9, 3):
        print(" ".join(map(str, state[i:i + 3])))

    print()


def reconstruct_path(parent, state):
    path = []

    while state is not None:
        path.append(state)
        state = parent[state]

    path.reverse()

    return path


def a_star(initial):

    priority_queue = []
```

```
    g_cost = {
        initial: 0
    }

    parent = {
        initial: None
    }

    g = 0
    h = manhattan_distance(initial)
    f = g + h

    heapq.heappush(
        priority_queue,
        (f, g, initial)
    )

    visited = set()

    while priority_queue:

        f, g, current = heapq.heappop(priority_queue)

        if current in visited:
            continue

        visited.add(current)

        if current == GOAL:
            return reconstruct_path(parent, current)

        for neighbor in get_neighbors(current):

            if neighbor in visited:
                continue

            new_g = g + 1

            if neighbor not in g_cost or new_g < g_cost[neighbor]:
```

```
                g_cost[neighbor] = new_g

                parent[neighbor] = current

                h = manhattan_distance(neighbor)

                f = new_g + h

                heapq.heappush(
                    priority_queue,
                    (f, new_g, neighbor)
                )

    return None


initial_state = (
    1, 3, 6,
    5, 0, 2,
    4, 7, 8
)

print("INITIAL STATE:")
print_puzzle(initial_state)

print("GOAL STATE:")
print_puzzle(GOAL)

solution = a_star(initial_state)

if solution:

    print("SOLUTION STEPS:")
    print()

    for step, state in enumerate(solution):

        print("Step", step)
        print_puzzle(state)
```

```
    print("Total moves:", len(solution) - 1)

else:

    print("No solution exists.")
```

```
INITIAL STATE:
1 3 6
5 0 2
4 7 8

GOAL STATE:
1 2 3
4 5 6
7 8 0

SOLUTION STEPS:

Step 0
1 3 6
5 0 2
4 7 8

Step 1
1 3 6
5 2 0
4 7 8

Step 2
1 3 0
5 2 6
4 7 8
```

```
Step 3
1 0 3
5 2 6
4 7 8

Step 4
1 2 3
5 0 6
4 7 8

Step 5
1 2 3
0 5 6
4 7 8

Step 6
1 2 3
4 5 6
0 7 8

Step 7
1 2 3
4 5 6
7 0 8

Step 8
1 2 3
4 5 6
7 8 0

Total moves: 8
```

—----------THE END —----------